

**EXP 6 IMPLEMENT QUICK SORT AND MERGE SORT AND OBSERVE THE
EXECUTION TIME FOR VARIOUS INPUT SIZES
(AVERAGE, WORST AND BEST CASES).**

AIM:

To implement Quick Sort and Merge Sort and observe their execution time for different input sizes in best, average, and worst cases.

Time Complexity Analysis

Sorting Algorithm	Best Case	Average Case	Worst Case
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

PROCEDURE:

1. Quick Sort

Quick Sort selects a **pivot** and divides the array into two parts:

- Elements smaller than pivot
- Elements greater than pivot

The process is repeated recursively.

- **Best case:** Pivot divides the array into two equal parts → **$O(n \log n)$**
- **Average case:** Partitions are reasonably balanced → **$O(n \log n)$**
- **Worst case:** Pivot is always the smallest or largest element → **$O(n^2)$**

2. Merge Sort

Merge Sort divides the array into two halves, sorts each half, and then merges them.

For every case, the array is divided into halves and merging takes $O(n)$ time.

Therefore:

- **Best case:** $O(n \log n)$
- **Average case:** $O(n \log n)$
- **Worst case:** $O(n \log n)$

OBSERVATION:

For different input sizes, record the execution time like this:

Input Size (n)	Quick Sort Best	Quick Sort Average	Quick Sort Worst	Merge Sort Best	Merge Sort Average	Merge Sort Worst
1000	—	—	—	—	—	—
5000	—	—	—	—	—	—
10000	—	—	—	—	—	—
20000	—	—	—	—	—	—

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Quick Sort

int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (a[j] <= pivot)
        {
            i++;

            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    int temp = a[i + 1];
    a[i + 1] = a[high];
    a[high] = temp;

    return i + 1;
}

void quickSort(int a[], int low, int high)
{
    if (low < high)
```

```

    {
        int p = partition(a, low, high);

        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

// Merge Sort

void merge(int a[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = 0;

    int *temp = (int *)malloc((high - low + 1) * sizeof(int));

    while (i <= mid && j <= high)
    {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= high)
        temp[k++] = a[j++];

    for (i = low, k = 0; i <= high; i++, k++)
        a[i] = temp[k];

    free(temp);
}

void mergeSort(int a[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
    }
}

```

```

        merge(a, low, mid, high);
    }
}

// INPUT GENERATION

/* Best case for Quick Sort:
Creates an input where the chosen last-element pivot is approximately the middle value.
*/
void quickBest(int a[], int low, int high)
{
    if (low > high)
        return;

    int mid = (low + high) / 2;

    static int value = 1;

    a[mid] = value++;

    quickBest(a, low, mid - 1);
    quickBest(a, mid + 1, high);
}

/* Average case: Random values */
void averageCase(int a[], int n)
{
    for (int i = 0; i < n; i++)
        a[i] = rand();
}

/* Worst case for our Quick Sort: Already sorted array.
Since the last element is selected as pivot, this produces highly unbalanced partitions.
*/
void quickWorst(int a[], int n)
{
    for (int i = 0; i < n; i++)
        a[i] = i + 1;
}

/* Merge Sort works in  $O(n \log n)$  for all cases. We still create three different input
arrangements. so that we can measure all three cases.
*/
void mergeBest(int a[], int n)
{

```

```
    for (int i = 0; i < n; i++)
        a[i] = i + 1;
}

void mergeAverage(int a[], int n)
{
    for (int i = 0; i < n; i++)
        a[i] = rand();
}

void mergeWorst(int a[], int n)
{
    for (int i = 0; i < n; i++)
        a[i] = n - i;
}

// TIME FUNCTIONS

double measureQuick(int a[], int n)
{
    clock_t start, end;

    start = clock();

    quickSort(a, 0, n - 1);

    end = clock();

    return (double)(end - start) / CLOCKS_PER_SEC;
}

double measureMerge(int a[], int n)
{
    clock_t start, end;

    start = clock();

    mergeSort(a, 0, n - 1);

    end = clock();

    return (double)(end - start) / CLOCKS_PER_SEC;
}

// Main
```

```

int main()
{
    int sizes[] = {1000, 5000, 10000, 20000};
    int numberOfSizes = 4;

    srand((unsigned int)time(NULL));

    printf("\n");

    printf("=====
=====\\n");
    printf("          SORTING EXECUTION TIME          \\n");

    printf("=====
=====\\n");

    printf("%-10s %-11.6s %-11.6s %-11.6s %-11.6s %-11.6s %-11.6s\\n",
        "Input(n)", "Quick", "Quick", "Quick", "Merge", "Merge", "Merge");
    printf("%-10s %-11.6s %-11.8s %-11.6s %-11.6s %-11.8s %-11.6s\\n",
        "          ", "Best", "Average", "Worst", "Best", "Average", "Worst");
    printf("-----\\n");

    for (int s = 0; s < numberOfSizes; s++)
    {
        int n = sizes[s];

        int *a = (int *)malloc(n * sizeof(int));

        if (a == NULL)
        {
            printf("Memory allocation failed!\\n");
            return 1;
        }

        double qb, qa, qw;
        double mb, ma, mw;

        /* ----- QUICK SORT BEST ----- */

        /*
        For simplicity, use a balanced permutation.
        This gives a good/best-case-like input.
        */
        for (int i = 0; i < n; i++)
            a[i] = i + 1;

```

```

/* Shuffle to avoid sorted input */
for (int i = n - 1; i > 0; i--)
{
    int j = rand() % (i + 1);

    int temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}

qb = measureQuick(a, n);

/* ----- QUICK SORT AVERAGE ----- */

averageCase(a, n);

qa = measureQuick(a, n);

/* ----- QUICK SORT WORST ----- */

quickWorst(a, n);

qw = measureQuick(a, n);

/* ----- MERGE SORT BEST ----- */

mergeBest(a, n);

mb = measureMerge(a, n);

/* ----- MERGE SORT AVERAGE ----- */

mergeAverage(a, n);

ma = measureMerge(a, n);

/* ----- MERGE SORT WORST ----- */

mergeWorst(a, n);

mw = measureMerge(a, n);

/* ----- PRINT RESULTS ----- */

printf("%-10d %-11.6f %-11.6f %-11.6f %-11.6f %-11.6f %-11.6f \n",
        n, qb, qa, qw, mb, ma, mw);

```

```

        free(a);
    }

    printf("=====\n");

    printf("\nTime Complexity:\n");
    printf("Quick Sort : Best = O(n log n), Average = O(n log n), Worst = O(n^2)\n");
    printf("Merge Sort : Best = O(n log n), Average = O(n log n), Worst = O(n log n)\n");

    return 0;
}

```

OUTPUT:

D:\>gcc Quickmerge.c -o Quickmerge.exe

D:\>Quickmerge.exe

=====

SORTING EXECUTION TIME

=====

Input(n)	Quick Best	Quick Average	Quick Worst	Merge Best	Merge Average	Merge Worst

1000	0.001000	0.000000	0.011000	0.000000	0.001000	0.001000
5000	0.002000	0.003000	0.262000	0.003000	0.004000	0.003000
10000	0.005000	0.005000	1.018000	0.007000	0.009000	0.006000
20000	0.010000	0.011000	4.091000	0.012000	0.019000	0.012000

=====

Time Complexity:

Quick Sort : Best = $O(n \log n)$, Average = $O(n \log n)$, Worst = $O(n^2)$

Merge Sort : Best = $O(n \log n)$, Average = $O(n \log n)$, Worst = $O(n \log n)$

```

D:\>gcc Quickmerge.c -o Quickmerge.exe
D:\>Quickmerge.exe
=====
                        SORTING EXECUTION TIME
=====
Input<n>    Quick      Quick      Quick      Merge      Merge      Merge
            Best       Average    Worst      Best       Average    Worst
-----
1000        0.001000    0.000000    0.011000    0.000000    0.001000    0.001000
5000        0.002000    0.003000    0.262000    0.003000    0.004000    0.003000
10000       0.005000    0.005000    1.018000    0.007000    0.009000    0.006000
20000       0.010000    0.011000    4.091000    0.012000    0.019000    0.012000
=====
Time Complexity:
Quick Sort   : Best =  $O(n \log n)$ , Average =  $O(n \log n)$ , Worst =  $O(n^2)$ 
Merge Sort   : Best =  $O(n \log n)$ , Average =  $O(n \log n)$ , Worst =  $O(n \log n)$ 

```

RESULT:

Merge Sort provides consistent $O(n \log n)$ performance for all cases, while Quick Sort is usually $O(n \log n)$ but can become $O(n^2)$ in the worst case.